Methodology for Generating Nested Scale Factors from the IMDb Non-Commercial Dataset for MongoDB Experiments

Overleaf-ready version in English

Prepared from the full `nhondangcode/imdb-non-commercial-datasets` snapshot and the complete JupyterLab/DuckDB workflow used in the experiments

1 Purpose of the Methodology

This document describes the full methodology used to prepare the IMDb dataset for experiments on the impact of EER conceptual-model elements and workload characteristics on query performance in a document data model using MongoDB.

The workflow had four main goals:

1. use the full IMDb snapshot available in the selected Kaggle package;
2. generate three nested scale factors, namely **SF0.25**, **SF0.5**, and **SF1**;
3. preserve the main title-level distributions as much as possible;
4. maintain cross-file consistency across the seven IMDb source files.

The final design sampled the data at the **title level** (`tconst`) and then induced the remaining files from the sampled set of titles. This decision was central to preserving the semantic structure of the dataset.

2 Dataset and Source Files

The experiments used the complete snapshot of the following seven IMDb files:

File	Role in the experimental dataset
`name.basics.tsv`	person-level attributes identified by `nconst`
`title.akas.tsv`	alternative titles, regions, and languages for each title
`title.basics.tsv`	core title metadata and the central source for `tconst`
`title.crew.tsv`	directors and writers associated with each title
`title.episode.tsv`	episode-to-parent-series relationships
`title.principals.tsv`	title-to-person participation records
`title.ratings.tsv`	average rating and number of votes for rated titles

Table 1: IMDb source files used in the workflow.

The complete title universe extracted from `title.basics.tsv` and materialized in the derived table `title_features` contained:

11,918,372 titles

This value was used as the basis for SF1.

3 Why Sampling Was Performed at the Title Level

The scale-factor generation did **not** sample each file independently. Instead, sampling was performed only on the set of titles (`tconst`), and all other files were induced afterward.

This choice was made for methodological reasons:

1. `title.basics.tsv` is the conceptual center of the IMDb subset;
2. `title.principals.tsv`, `title.crew.tsv`, `title.ratings.tsv`, `title.akas.tsv`, and `title.episode.tsv` all depend on titles;
3. independent file-level sampling would create incomplete titles, broken person references, and structurally inconsistent subsets;
4. title-level sampling is more compatible with MongoDB-oriented experiments, because titles naturally serve as the main document anchor for embedding or referencing decisions.

Therefore, the adopted rule was:

> First sample titles, then derive the remaining files from the sampled title set.

4 Execution Environment and Parsing Strategy

The workflow was implemented in **JupyterLab** with **DuckDB**. DuckDB was preferred over loading all files directly into pandas because the full snapshot was large and some IMDb files contained parsing edge cases.

4.1 Reason for the DuckDB-based ingestion step

Two source files, especially `title.akas.tsv` and `title.principals.tsv`, contained text fields with literal double quotes and unusually long lines. These properties caused repeated CSV/TSV auto-detection failures in standard loading routines.

To make ingestion robust, the TSV import step adopted the following strategy:

1. read the header in Python;
2. explicitly define all columns as `VARCHAR`;
3. disable schema auto-detection;
4. disable quote and escape semantics;
5. disable parallel CSV reading;
6. increase the maximum allowed line size.

Conceptually, the loading policy was:

> Treat IMDb files as raw tab-separated text first, and only cast to typed values later in derived tables.

This was essential for successfully materializing all seven views.

5 Construction of the Title-Level Feature Table

After loading the seven IMDb files, the workflow created intermediate aggregates and then joined them into a single title-level table named `title_features`. This table contained one row per `tconst` and served as the basis for stratification.

5.1 Intermediate title-level aggregates

The following aggregate tables were produced:

- `principals_agg`: number of participation rows, number of distinct people, and number of distinct principal categories per title;
- `akas_agg`: number of alternate titles, number of distinct regions, and number of distinct languages per title;
- `crew_agg`: number of directors and number of writers per title;
- `ratings_agg`: rating attributes and a `has_rating` flag;
- `episode_agg`: `parentTconst`, season number, episode number, and an `is_episode` flag.

5.2 Main features materialized for each title

The resulting `title_features` table included, among others, the following derived attributes:

Feature	Meaning
`titleType`	original IMDb title type
`startYear`, `endYear`	temporal information for the title
`runtimeMinutes`	running time when available
`genres`	raw IMDb genre string
`primaryGenre`	first genre extracted from the IMDb genre list
`decade`	decade derived from `startYear`
`is_episode`	flag indicating whether the title is an episode
`parentTconst`	parent series for episode records
`principals_count`	number of principal rows for the title
`akas_count`	number of alternate titles
`directors_count`, `writers_count`	crew cardinalities
`has_rating`, `averageRating`, `numVotes`	rating-related attributes

Table 2: Main title-level features used for stratified sampling.

6 Bucket Construction for Stratification

The methodology did not stratify directly on raw numeric values. Instead, it converted selected variables into buckets to avoid over-fragmenting the strata.

6.1 Principal-cardinality bucket

The bucket for the number of principal records was defined as:

$$\texttt{principals_bucket} = \begin{cases} 0 & \text{if } principals_count = 0 \\ 1 & \text{if } principals_count = 1 \\ 2_3 & \text{if } 2 \leq principals_count \leq 3 \\ 4_6 & \text{if } 4 \leq principals_count \leq 6 \\ 7_10 & \text{if } 7 \leq principals_count \leq 10 \\ 11_plus & \text{if } principals_count \geq 11 \end{cases}$$

6.2 Vote-volume bucket

The vote-volume bucket was defined as:

$$\texttt{numVotes_bucket} = \begin{cases} 0 & \text{if } numVotes = 0 \text{ or null} \\ 1_9 & \text{if } 1 \leq numVotes \leq 9 \\ 10_99 & \text{if } 10 \leq numVotes \leq 99 \\ 100_999 & \text{if } 100 \leq numVotes \leq 999 \\ 1000_9999 & \text{if } 1000 \leq numVotes \leq 9999 \\ 10000_plus & \text{if } numVotes \geq 10000 \end{cases}$$

6.3 Runtime bucket

The runtime bucket was defined as:

$$\texttt{runtime_bucket} = \begin{cases} unknown & \text{if runtime is null} \\ lt_10 & \text{if runtime} < 10 \\ 10_29 & \text{if } 10 \leq runtime < 30 \\ 30_59 & \text{if } 30 \leq runtime < 60 \\ 60_119 & \text{if } 60 \leq runtime < 120 \\ 120_plus & \text{if runtime} \geq 120 \end{cases}$$

7 Definition of the Stratification Key

Each title received a composite stratum key built from a combination of categorical and bucketed attributes. The selected key was:

$$\texttt{stratum_key} = (titleType,\ decade,\ primaryGenre,\ hasRating,\ principalsBucket)$$

This choice was intended to preserve the following aspects simultaneously:

1. semantic type of title;

2. temporal distribution;
3. genre composition;
4. presence or absence of ratings;
5. cardinality of title-to-person participation.

This combination was considered a good compromise between representational fidelity and computational tractability.

8 Nested Sampling Procedure

8.1 Target scale factors

The methodology defined three scale factors:

- **SF0.25**
- **SF0.5**
- **SF1**

The sampling was designed to be **nested**, that is:

$$SF0.25 \subset SF0.5 \subset SF1$$

8.2 Reproducible ordering within each stratum

Within each stratum, titles were ordered using a deterministic hash of the title identifier combined with a fixed seed. A row number was then assigned inside each stratum:

$$rn_i = row_number\ within\ stratum$$

and the stratum size was also computed:

$$n_s = \text{number of titles in stratum } s$$

8.3 Selection rule

For each stratum s, the quotas were computed approximately as:

$$q_{0.25}(s) = \max(1,\ round(0.25 \cdot n_s))$$

$$q_{0.5}(s) = \max(1,\ round(0.5 \cdot n_s))$$

A title entered the scale factors according to:

- `in_sf_025 = 1` if $rn_i \leq q_{0.25}(s)$
- `in_sf_05 = 1` if $rn_i \leq q_{0.5}(s)$
- `in_sf_1 = 1` for all titles

Because the rank order was shared, nesting was preserved by construction.

9 Episode-to-Series Referential Closure

The title-level sample was then expanded with a closure rule for episodes:

> If an episode entered a scale factor, its parent series had to be included as well.

This step was necessary because `title.episode.tsv` contains hierarchical episode-to-series relationships. Without closure, sampled episodes could reference a parent series absent from the same subset.

The closure was applied by adding `parentTconst` from episode records to the sampled title set, while retaining only parent titles that actually existed in the title universe.

9.1 Observed consequence of closure

This repair step slightly changed the nominal proportions of some title types, especially `tvSeries`, because many sampled episodes pulled their parent series into the final title set.

This is visible in the experimental distributions reported later in this document.

10 Scale-Factor Sizes Obtained

After nested sampling and parent-series closure, the title counts were:

Scale factor	Number of titles
SF0.25	3,119,041
SF0.5	6,067,596
SF1	11,918,372

Table 3: Final title counts after title-level sampling and episode-parent closure.

11 Distributional Validation

Two validation views were inspected before export:

1. title-type distribution;
2. primary-genre distribution.

The objective was not to obtain exact proportional equality, but rather to preserve the overall profile of the original dataset while keeping the subsets nested and relationally coherent.

11.1 Title-type distribution

titleType	n_full	n_025	n_05
tvEpisode	9,177,015	2,294,688	4,589,138
short	1,083,730	271,594	542,630
movie	725,936	182,165	363,761
video	316,291	79,459	158,602
tvSeries	287,707	186,514	230,194
tvMovie	152,375	38,623	76,753
tvMiniSeries	65,020	37,283	50,310
tvSpecial	54,310	14,058	27,591
videoGame	45,226	11,637	22,947
tvShort	10,761	3,018	5,668
tvPilot	1	1	1

Table 4: Title-type distribution for the full title universe and the two reduced scale factors.

The most relevant observation is the inflation of `tvSeries` in SF0.25 and SF0.5 relative to the nominal rates. This was expected and directly caused by the episode-parent closure step.

11.2 Primary-genre distribution

primaryGenre	n_full	n_025	n_05
Drama	2,500,976	644,278	1,265,005
Comedy	1,939,545	513,744	991,861
Documentary	988,009	263,677	507,636
News	958,584	242,719	481,688
Talk-Show	750,871	194,874	380,918
unknown	527,273	139,654	269,788
Action	498,162	131,411	254,243
Reality-TV	410,393	111,257	211,833
Adult	382,392	97,500	192,644
Animation	371,605	99,478	190,835
Crime	356,773	94,507	182,481
Family	356,742	93,155	181,416
Game-Show	287,677	75,042	146,210
Adventure	280,595	74,698	143,754
Short	277,145	70,061	139,240
Music	222,472	58,653	113,699
Sport	178,636	47,054	91,227
Romance	135,719	35,381	68,996
Biography	116,729	31,063	59,895

primaryGenre	n_full	n_025	n_05
Horror	114,665	29,790	58,302

Table 5: Top primary-genre distribution for the full title universe and the two reduced scale factors.

The genre distribution remained much closer to the expected 25% and 50% proportions than the title-type distribution. This suggests that title-level stratification was effective in preserving the global genre profile.

12 Induction of the Remaining Files

Once the final title sets were defined, the remaining files were derived as follows:

- `title.basics.tsv`: all titles in the selected scale factor;
- `title.akas.tsv`: all rows whose `titleId` belongs to the selected titles;
- `title.crew.tsv`: all rows whose `tconst` belongs to the selected titles;
- `title.episode.tsv`: all episode rows whose `tconst` belongs to the selected titles;
- `title.principals.tsv`: all rows whose `tconst` belongs to the selected titles;
- `title.ratings.tsv`: all rows whose `tconst` belongs to the selected titles.

The only remaining file requiring a separate induction procedure was `name.basics.tsv`, because person records are identified by `nconst` and must be derived from person references that appear in title-dependent files.

13 Person Induction for `name.basics.tsv`

The person universe for each scale factor was built as the union of:

1. all `nconst` values referenced by `title.principals.tsv`;
2. all directors and writers extracted from `title.crew.tsv`.

This produced three person sets:

- `sf_025_people`
- `sf_05_people`
- `sf_1_people`

14 Validation Against the Original Snapshot

During validation, two important anomalies were detected:

1. `title.principals.tsv` referenced many `nconst` values not present in `name.basics.tsv`;
2. `title.episode.tsv` contained a very small number of parent-series references not present in `title.basics.tsv`.

The following counts were measured directly on the original snapshot:

Original anomaly	Count
Rows in `title.principals` whose `nconst` is missing from `name.basics`	23,562,895
Rows in `title.episode` whose `parentTconst` is missing from `title.basics`	2

Table 6: Inconsistencies already present in the original IMDb snapshot.

These results are important because they show that the inconsistencies were not introduced by the scale-factor generation procedure.

15 Repair Strategy Applied at Export Time

Because the goal was to obtain experimentally usable subsets with stronger referential consistency than the raw snapshot, two lightweight repairs were applied at export time.

15.1 Repair 1: eliminate orphan episode-parent references

For `title.episode.tsv`, only rows whose `tconst` and `parentTconst` both existed in the corresponding scale factor and in `title.basics.tsv` were retained. This removed the few orphan cases inherited from the source.

One residual problematic parent identifier was observed during debugging:

`tt38353028`

This identifier was traced to the source inconsistency pattern and motivated the final stricter export join against `title.basics.tsv`.

15.2 Repair 2: add stub records to `name.basics.tsv`

Because removing all broken `title.principals` references would have discarded a very large amount of participation information, the chosen solution was to augment `name.basics.tsv` with stub person records. Each stub kept only `nconst` and left the remaining attributes null.

The number of stub records required per scale factor was:

Scale factor	Number of stub person records
SF0.25	1,250,527
SF0.5	1,799,503
SF1	2,597,537

Table 7: Stub records added to `name.basics.tsv` to repair missing person references.

16 Post-Repair Validation

After adding stub person records, the validation results showed:

- `principals_name_missing = 0` for SF0.25;
- `principals_name_missing = 0` for SF0.5;
- `principals_name_missing = 0` for SF1.

Before the final stricter re-export of `title.episode.tsv`, the intermediate light validation reported:

- `episode_parent_missing = 1` for SF0.25;
- `episode_parent_missing = 1` for SF0.5;
- `episode_parent_missing = 0` for SF1.

These remaining cases were traced to the source-side orphan-parent anomaly and motivated the final export condition that simultaneously checks membership in the scale-factor title set and existence in `title.basics.tsv`.

17 Methodological Interpretation

The adopted workflow has several strengths for MongoDB experiments:

1. the scale factors are conceptually consistent because titles are sampled first and dependent files are induced afterward;
2. the subsets are nested, which supports fairer comparisons across volumes;
3. the main title-level distributions are reasonably preserved;
4. the procedure is reproducible because it uses deterministic ranking within strata;
5. source inconsistencies are handled explicitly rather than silently ignored.

The most important limitation is that perfect proportional preservation of all title types is incompatible with strict episode-parent closure. In practice, this causes the number of `tvSeries` titles to rise above the nominal 25% and 50% targets in the reduced subsets. However, this deviation is justified because preserving hierarchical consistency is more important than preserving exact type proportions.

18 Recommended Wording for the Paper

The following paragraph can be used almost directly in the paper:

> We generated SF0.25 and SF0.5 as nested stratified subsets of a frozen SF1 IMDb snapshot. Sampling was performed at the title level (`tconst`), using strata defined by title type, release decade, primary genre, rating availability, and principal-cast cardinality. The remaining files were induced from the sampled titles. Referential closure was then applied so that sampled episodes retained their parent series. During validation, we found that the original snapshot already contained missing person references in `title.principals` and a very small number of orphan parent-series references in `title.episode`. To obtain referentially usable subsets, we removed orphan episode-parent rows and augmented `name.basics` with stub person records containing only `nconst`.

19 Final Workflow Summary

The full process can be summarized as follows:

1. load the seven IMDb TSV files into DuckDB using robust text-first parsing;
2. create aggregate tables from `title.principals`, `title.akas`, `title.crew`, `title.episode`, and `title.ratings`;
3. build `title_features` with one row per title;
4. derive buckets and a composite stratification key;
5. rank titles deterministically inside each stratum;
6. generate nested title sets for SF0.25, SF0.5, and SF1;
7. apply episode-to-series closure;
8. derive all title-dependent files from the sampled titles;
9. derive person sets from `title.principals` and `title.crew`;
10. export `name.basics.tsv` with additional stub records for missing `nconst`;
11. re-export `title.episode.tsv` with stricter parent validation;
12. validate the exported subsets.

20 Conclusion

The resulting methodology produces three reproducible and nested IMDb scale factors suitable for MongoDB performance experiments driven by conceptual modeling decisions and workload characteristics. The approach preserves the title-centered semantics of the dataset, keeps the main distributions reasonably stable, and explicitly handles inconsistencies inherited from the original snapshot.

As a result, the generated subsets are substantially better suited for controlled document-model experiments than naive file-level sampling or arbitrary row truncation.